Разбор контрольной

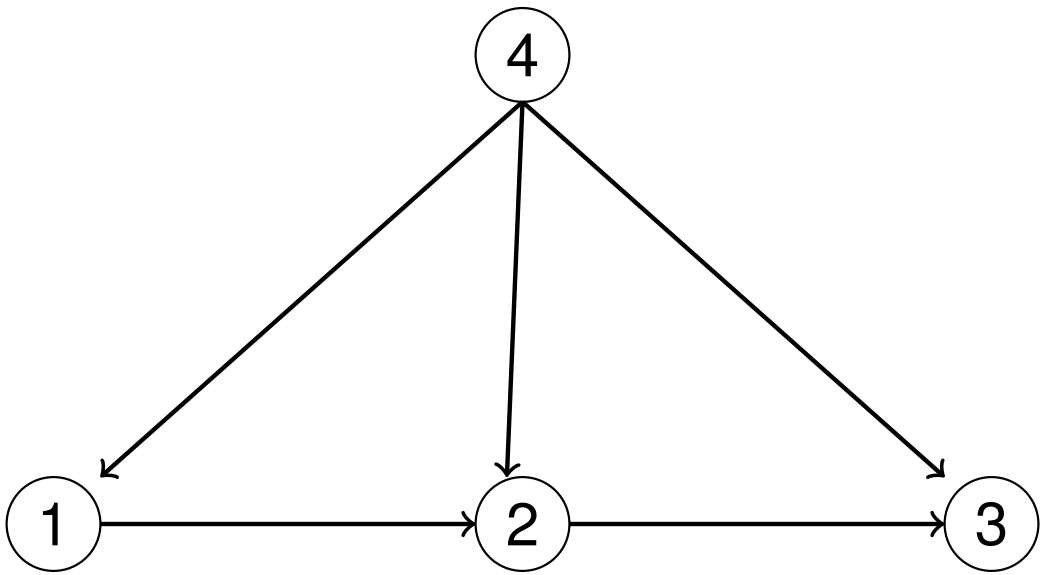
Юрий Литвинов
y.litvinov@spbu.ru

21.03.2026

Разбор летучки

Задача 1, про матрицу инцидентности

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & -1 \\ -1 & -1 & 1 & 0 & 0 \\ 0 & 0 & -1 & -1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix}$$



Разбор летучки

Задача 2, про трудоёмкость BFS в худшем случае

- $O(|V| + |E|)$
- BFS запоминает посещённые вершины и посещает каждую ровно один раз, но должен просмотреть все рёбра от каждой вершины
- Худший случай — если граф полносвязный и надо обойти его весь, трудоёмкость такая же
- $|E| \in O(|V|^2)$, так что общая трудоёмкость получится $O(|V|^2)$, что не противоречит написанному выше
- Ответ без пояснений полностью не засчитывался

Разбор летучки

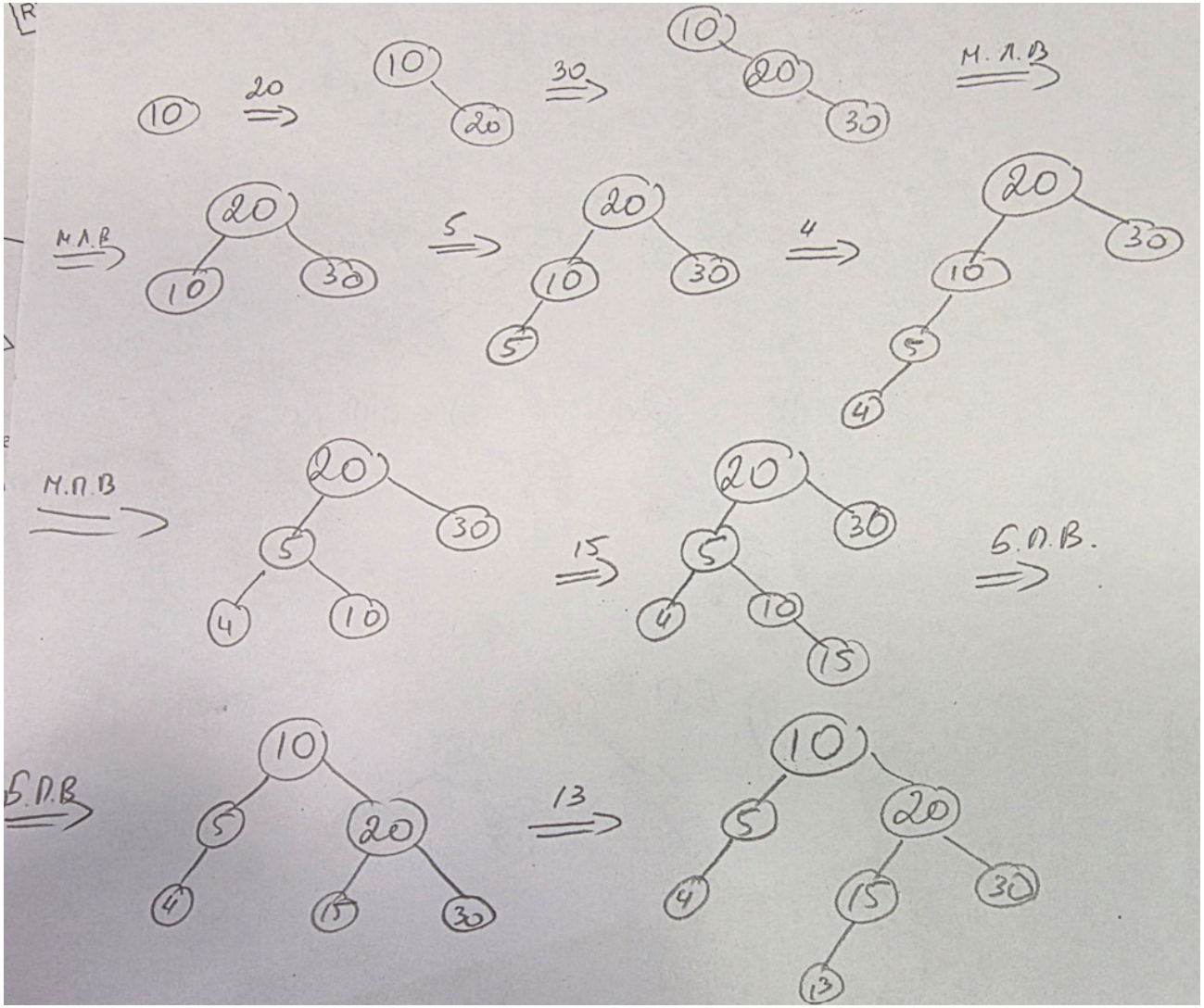
Задача 3, про функцию neighbours

```
void neighbours(int v, int size, int** graph) {
    if (v < 0 || v >= size || graph == NULL || *graph == NULL) {
        return;
    }

    for (int i = 0; i < size; ++i) {
        if (graph[v][i]) {
            printf("%d ", i);
        }
    }
}
```

Разбор контрольной

Задача 1, АВЛ-дерево



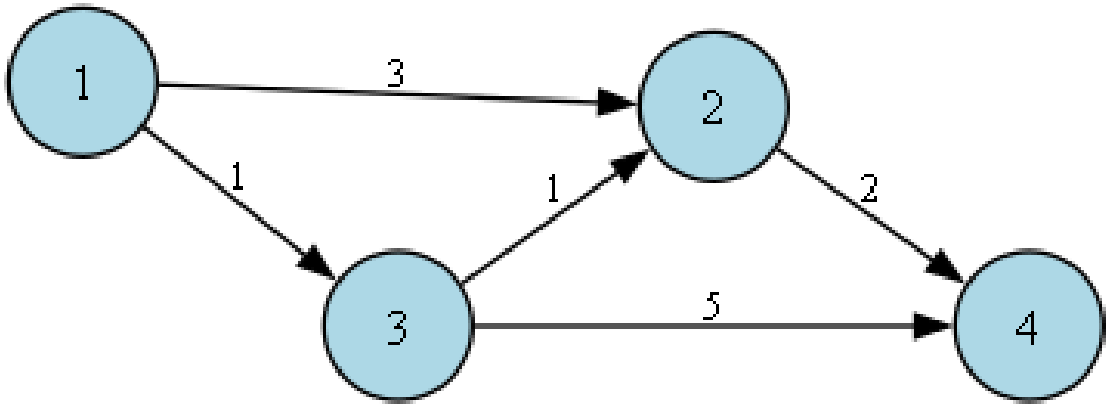
Разбор контрольной

Задача 2, Дейкстра

Матрица смежности:

$$\begin{pmatrix} & 1 & 2 & 3 & 4 \\ 1 & - & 3 & 1 & - \\ 2 & - & - & - & 2 \\ 3 & - & 1 & - & 5 \\ 4 & - & - & - & - \end{pmatrix}$$

Визуализация графа:



Разбор контрольной

Задача 2, Дейкстра

Таблица:

Шаг	Текущая вершина	d[1]	d[2]	d[3]	d[4]
Начало	—	0	∞	∞	∞
1	1	0	3 (1)	1 (1)	∞
2	3	0	2 (3)	1 (1)	6 (3)
3	2	0	2 (3)	1 (1)	4 (2)
4	4	0	2 (3)	1 (1)	4 (2)

Путь: $1 \to 3 \to 2 \to 4$, длина 4

Разбор контрольной

Задача 2, Дейкстра

Проблемы:

- Многие забыли указать путь и/или длину пути
- На шаге 2 в d[4] оказывались значения от 5 до 8

Разбор контрольной

Задача 3, высота узла

```
int nodeHeight(Node *root, int key) {
   if (root == NULL) return -1;

   if (key < root->key)
      return nodeHeight(root->left, key);
   if (key > root->key)
      return nodeHeight(root->right, key);

   return height(root);
}

int height(Node *node) {
   if (node == NULL) return -1;
   int leftH  = height(node->left);
   int rightH = height(node->right);
   return (leftH > rightH ? leftH : rightH) + 1;
}
```

Разбор контрольной

Задача 3, высота узла

Ещё одно решение, без вспомогательной функции:

```
int nodeHeight(Node *root, int key) {
   if (root == NULL) return -1;

   if (key < root->key)
      return nodeHeight(root->left, key);
   if (key > root->key)
      return nodeHeight(root->right, key);

   int leftH  = node->left ? nodeHeight(node->left, node->left->key) : -1;
   int rightH = node->right ? nodeHeight(node->right, node->right->key) : -1;
   return (leftH > rightH ? leftH : rightH) + 1;
}
```

Разбор контрольной

Задача 4, сложность Дейкстры

- Наивная реализация: $O(|V|^2 + |E|) = O(|V|^2)$, — мы при посещении каждой вершины ищем ближайшую к стартовой за линию (граф может быть полносвязным) и обновляем длины путей, проходясь по рёбрам (не более одного раза каждое)
- С кучей: $O(|V| log(|V|) + |E| log(|V|))$ — извлечение минимума из кучи за логарифм, обновление расстояния за логарифм (вставка в кучу) для каждого исходящего ребра (всего $|E|$ раз)
- Многие написали ответ, но не пояснили, почему
- Многие забыли про рёбра

Разбор контрольной

Задача 5, умножение матриц

a) Массивы хранятся в row-major порядке: B[k][0], B[k][1], ...лежат подряд, B[0][j], B[1][j], ...— вразброс, так что в matmul_v1 (где обход B в column-major) происходят промахи кеша. В matmul_v2 все матрицы читаются в row-major, кеш используется максимально

b) Кеш-линии обычно 64 байта, при последовательном доступе одно чтение на 16 значений из матрицы, при column-major — на одно

c) Проверить можно
perf stat -e cache-misses,cache-references ./matmul_v1 (или matmul_v2)

- Требовалось внятное объяснение, хотя бы упоминающее кеш и последовательность доступа
- Никто не ожидал ключи для perf, но надо было написать, что именно смотреть для проверки гипотезы